

Technical Documentation for Drift Detection

March 22, 2026

Contents

1	Drift detection	1
1.1	Objective, scope, and relation to the paper	1
1.2	Dependencies and data sources	2
1.3	Tab-by-tab UI map	2
1.4	Feature engineering and target construction	3
1.5	Data preparation: Stage 1 and Stage 2	6
1.6	Feature selection	7
1.7	Experimental design and strategies	7
1.8	Metrics, tables, and final artifacts	11
1.9	Validation through tests	12

1 Drift detection

1.1 Objective, scope, and relation to the paper

The module `src/drift_detection_app.py` implements a replication workspace for the paper *Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining*. The file itself states that the full *Drift detection* section is intentionally hosted in a single Python source, and it concentrates methodology replication, feature construction, feature selection, experiment execution, and coverage verification.

The real scope of the module goes beyond a single drift detector. It includes:

- deterministic crash-prediction dataset preparation for the selected corridor;
- yearly batch recalibration strategies;
- adaptive strategies based on ADWIN, ARF, and KSWIN;
- persistence for checkpoints, tuning caches, and SMOTE artifacts;
- reporting layers aligned with Appendix A.6–A.9 and Figure 7 of the paper.

The reference paper setup is fixed through `PAPER_TIME_SPAN = 2018-01-01 to 2024-09-30`. In addition, the focus corridor is constrained to the fixed gantry set 11, 12, 14, 15 (`DRIFT_FOCUS_PORTICOS`), while the segment order used by the article feature engineering follows 15→14, 14→12, and 12→11.

1.2 Dependencies and data sources

The module assumes `Datos/flujos.duckdb` as the main flow source and `flujos_duckdb` as the input table. For events it uses CSV files under `Datos/`, processed with `process_accidentes_df()`, and it relies on `Datos/Porticos.csv` for geospatial mapping and gantry relationships via `load_porticos()`, `find_candidate_porticos()`, and `get_portico_segments()`.

The app also depends on optional libraries depending on the stage:

- `duckdb` for feature querying and persistence;
- `optuna` for hyperparameter tuning;
- `imblearn` for SMOTE;
- `river` for ADWIN, KSWIN, and `AdaptiveRandomForestClassifier`.

1.3 Tab-by-tab UI map

The `main()` function creates six tabs with the actual labels `Blueprint`, `Eventos`, `Feature engineering`, `Feature Selection`, `Experiments`, and `Coverage`. Each tab corresponds to an explicit functional block:

Blueprint `_render_blueprint_tab()` shows the article replication matrix, the related-work table (`build_related_work_table()`), covered sections/figures/tables, and the migration plan from R to Python (`build_python_migration_review_plan()`).

Eventos `_render_events_tab()` loads one or multiple accident files from `Datos/`, assigns them to gantries with `process_accidentes_df()`, preserves excluded events when no mapping is possible, and builds a corridor-level summary that mirrors *Crash prediction*.

Feature engineering `_render_feature_engineering_tab()` offers three sources: load an existing DuckDB, recompute features, or reuse the in-memory dataset. In computation mode, it queries `flujos.duckdb`, generates gantry/segment/lane variables, builds `target`, applies Stage 1 and Stage 2, and exports a DuckDB containing `raw_features` and `clean_features`.

Feature Selection `_render_feature_selection_tab()` runs correlation filtering, Random-Forest ranking, and persistence of the final feature-selection payload inside the same feature DuckDB file.

Experiments `_render_experiments_tab()` locks the final feature set, allows the user to select batch models, strategies, ARF/KSWIN variants, balance modes, and sequential seeds, and then invokes `run_recalibration_experiments()` with support for checkpoint preview, resume, and fresh-start execution.

Coverage `_render_coverage_tab()` validates that all sections, figures, and tables declared in `ARTICLE_SECTIONS`, `ARTICLE_FIGURES`, and `ARTICLE_TABLES` map to concrete Python artifacts. This is summarized through `article_coverage_percentage()`.

1.4 Feature engineering and target construction

Feature engineering is centered on `_compute_drift_article_features()`, supported by `feature_catalog_table()` and `feature_engineering_reference_table()`. The goal is to replicate Table 4 of the paper and produce a prediction-ready dataset on a 5-minute time grid while preserving the exact operational semantics of the code.

Operational notation Let $\Delta = 5$ minutes denote the interval width, and define

$$b(\tau) = \left\lfloor \frac{\tau}{\Delta} \right\rfloor \Delta$$

as the function that maps a timestamp τ to the start of its interval. Under the default article configuration, the corridor uses the gantry order

$$P = (15, 14, 12, 11),$$

the category set

$$C = \{\text{Light}, \text{Heavy}, \text{Motorcycle}\},$$

and the consecutive segments

$$S = \{(15, 14), (14, 12), (12, 11)\}.$$

Each raw AVI record contributes a timestamp τ_r , point speed v_r , category c_r , license plate π_r , gantry p_r , and lane ℓ_r . The app maps `CATEGORIA` to `Light/Heavy/Motorcycle` through `DRIFT_ARTICLE_CATEGORY_MAP`, namely $\{1 \mapsto \text{Light}, 2 \mapsto \text{Heavy}, 3 \mapsto \text{Heavy}, 4 \mapsto \text{Motorcycle}\}$.

Generated feature space With the fixed article corridor of four gantries, three segments, three categories, and three lanes, the default configuration yields 260 numeric predictors before adding **target**:

- 96 gantry-by-category variables: 4 base metrics + 4 deltas, for 4 gantries and 3 categories;
- 8 compositional gantry variables: `ft_motorcycle_*` and `ft_heavy_*`;
- 72 segment-by-category variables: 5 base metrics + 3 deltas, for 3 segments and 3 categories;
- 21 segment-global variables: 4 base metrics + 3 deltas, for 3 segments;
- 63 origin-lane variables: 4 base metrics + 3 deltas, for 3 segments and 3 lanes.

If the user restricts gantries or categories in the UI, this space shrinks, but the mathematical definitions remain unchanged.

Gantry-by-category variables For each interval t , gantry $p \in P$, and category $c \in C$, define the observation set

$$G_{t,p,c} = \{r : b(\tau_r) = t, p_r = p, c_r = c\}.$$

The code then builds the `flow_`, `vel_`, `sd_`, and `den_` columns as

$$\text{Flow}_{t,p,c} = |G_{t,p,c}|,$$

$$\begin{aligned}
\text{Vel}_{t,p,c} &= \frac{1}{|G_{t,p,c}|} \sum_{r \in G_{t,p,c}} v_r, \\
\text{Sd}_{t,p,c} &= \sqrt{\frac{1}{|G_{t,p,c}| - 1} \sum_{r \in G_{t,p,c}} (v_r - \text{Vel}_{t,p,c})^2}, \\
\text{Den}_{t,p,c} &= \begin{cases} \frac{\text{Flow}_{t,p,c}}{\text{Vel}_{t,p,c}}, & \text{if } \text{Vel}_{t,p,c} > 0, \\ \text{NA}, & \text{otherwise.} \end{cases}
\end{aligned}$$

In the actual implementation, Flow is filled with 0 when no observations exist, whereas Vel, Sd, and Den remain missing when the group does not exist or lacks enough support. In addition, Sd uses the sample standard deviation from `pandas`; therefore, when only one vehicle is present, the value remains NaN.

Temporal changes are built through first differences inside each time series:

$$\Delta X_{t,p,c} = X_{t,p,c} - X_{t-\Delta,p,c}, \quad X \in \{\text{Flow}, \text{Vel}, \text{Den}, \text{Sd}\}.$$

This produces `delta_flow_*`, `delta_vel_*`, `delta_den_*`, and `delta_sd_*`. For the first interval where the base variable is defined, the code sets the delta to 0; if the base variable starts as missing, the delta also remains missing.

Compositional gantry shares At the same interval and gantry, the app computes the total flow

$$\text{Flow}_{t,p}^{\text{tot}} = \text{Flow}_{t,p,\text{Light}} + \text{Flow}_{t,p,\text{Heavy}} + \text{Flow}_{t,p,\text{Motorcycle}}.$$

It then derives the shares

$$\text{FtMotorcycle}_{t,p} = \frac{\text{Flow}_{t,p,\text{Motorcycle}}}{\text{Flow}_{t,p}^{\text{tot}}}, \quad \text{FtHeavy}_{t,p} = \frac{\text{Flow}_{t,p,\text{Heavy}}}{\text{Flow}_{t,p}^{\text{tot}}}.$$

These are the `ft_motorcycle_*` and `ft_heavy_*` columns. If the total gantry flow is zero, both shares remain NaN instead of being forced to zero, because the denominator does not represent an observed mixture.

Segment matching between consecutive gantries Segment features are not computed from local gantry speeds, but from matched trips between consecutive gantries. For each segment $s = (p^-, p^+) \in S$, the distance d_s is obtained through `_lookup_article_segment_distance_km()`, prioritizing roadway metadata and falling back to geometric estimates when needed.

The maximum matching window depends on segment distance:

$$\tau_s = \max\left(30, \frac{60 d_s}{20}\right) \text{ minutes},$$

because `DRIFT_ARTICLE_MIN_MATCH_WINDOW_MINUTES` = 30 and the minimum reference speed is 20 km/h. An upstream-downstream pair is accepted if:

- it shares the same plate π ;
- the downstream passage occurs after the upstream passage and within τ_s ;
- the category is consistent at both ends;

- travel time is strictly positive.

The implementation uses `merge_asof(..., direction="backward")` from the downstream gantry back to the upstream one, then removes duplicates by `plate_clean` and `time_start`, keeping the first compatible downstream passage for each upstream observation. The match interval is anchored to the upstream timestamp, that is, to $b(\tau_{\text{start}})$.

For each matched trip m on segment s , define the travel time

$$\text{TT}_m = \tau_m^{\text{end}} - \tau_m^{\text{start}},$$

and the spatial speed

$$\text{SegVel}_m = \begin{cases} \frac{d_s}{\text{TT}_m / 1 \text{ hour}}, & \text{if } d_s > 0, \\ \text{NA}, & \text{if no valid distance is available.} \end{cases}$$

In parallel, the lane-change indicator is

$$\text{LC}_m = \mathbf{1}\{\ell_m^{\text{start}} \neq \ell_m^{\text{end}}\},$$

whenever both lanes are known; otherwise the match contributes `NaN` to that component.

Segment-by-category variables Let $M_{t,s,c}$ denote the set of matches for interval t , segment s , and category c . From that set the code builds columns such as `flow_light_15_14`, `vel_heavy_14_12`, or `cl_motorcycle_12_11`:

$$\begin{aligned} \text{Flow}_{t,s,c} &= |M_{t,s,c}|, \\ \text{Vel}_{t,s,c} &= \frac{1}{|M_{t,s,c}|} \sum_{m \in M_{t,s,c}} \text{SegVel}_m, \\ \text{Sd}_{t,s,c} &= \sqrt{\frac{1}{|M_{t,s,c}| - 1} \sum_{m \in M_{t,s,c}} (\text{SegVel}_m - \text{Vel}_{t,s,c})^2}, \\ \text{CL}_{t,s,c} &= \frac{1}{|M_{t,s,c}^{LC}|} \sum_{m \in M_{t,s,c}^{LC}} \text{LC}_m, \\ \text{Den}_{t,s,c} &= \begin{cases} \frac{\text{Flow}_{t,s,c}}{\text{Vel}_{t,s,c}}, & \text{if } \text{Vel}_{t,s,c} > 0, \\ \text{NA}, & \text{otherwise.} \end{cases} \end{aligned}$$

Here $M_{t,s,c}^{LC}$ denotes the subset with observed lanes at both ends. The deltas ΔFlow , ΔVel , and ΔDen reuse the same first-difference definition as the gantry block. There is no ΔCL in the current implementation.

Segment-global and origin-lane variables The segment-global block aggregates all matches in the interval without splitting by category:

$$M_{t,s} = \bigcup_{c \in C} M_{t,s,c}.$$

On top of $M_{t,s}$ the code generates `flow_15_14`, `vel_15_14`, `sd_15_14`, `den_15_14`, and their temporal deltas. Mathematically, the formulas are identical to the segment-by-category block, except that $M_{t,s,c}$ is replaced by $M_{t,s}$.

To capture lane heterogeneity, the same matching output is re-aggregated by origin lane:

$$M_{t,s,\ell} = \{m \in M_{t,s} : \ell_m^{\text{start}} = \ell\}, \quad \ell \in \{1, 2, 3\}.$$

The app then computes $\text{Flow}_{t,s,\ell}$, $\text{Vel}_{t,s,\ell}$, $\text{Sd}_{t,s,\ell}$, $\text{Den}_{t,s,\ell}$, and their deltas, producing columns such as `vel_lane2_15_14` or `delta_den_lane3_12_11`. This block matters because the code explicitly motivates it by the empirical relevance of lane-specific variables in the Figure 6 replica.

Boundary rules and numerical consistency The implementation distinguishes between absence of traffic and absence of measurement:

- flows are filled with 0 when the group has no observations;
- speeds, standard deviations, densities, and lane-change fractions remain NaN when there is not enough information;
- deltas are computed on the already pivoted feature matrices, so they inherit the same missing-value pattern;
- when computation is done in DuckDB batches, static features match the one-pass computation, and deltas are reinitialized at batch boundaries with 0 or NaN; this is covered by `test_drift_batched_flow_features`

Target construction The target is added by `_add_interval_accident_target()` after corridor accidents are filtered with `_filter_accidents_for_allowed_porticos()`. If t_a is an accident timestamp, its binary label is assigned to the interval

$$t^* = b(t_a) - \Delta.$$

Therefore,

$$y_t = 1 \iff \exists a \text{ such that } b(t_a) = t + \Delta.$$

This definition implements a one-bin lead time: features at t aim to anticipate accidents occurring in the next 5-minute interval. The test `test_drift_article_features_cover_table4_and_target_shift` validates this shift explicitly; for example, an accident at 00:06 activates `target = 1` on the 00:00 row, not on the 00:05 row.

1.5 Data preparation: Stage 1 and Stage 2

Paper Section 4.2 is operationalized through `apply_missing_data_policy()`, `identify_long_zero_accident_runs()`, `filter_long_zero_accident_runs()`, and `run_configurable_preparation_pipeline()`.

- **Stage 1:** remove variables with more than 1% missing values, then drop intervals with any missing predictor among the remaining variables.
- **Stage 2:** remove long accident-free windows, by default at least 7 consecutive days with `target = 0`.

The output includes a `DataPreparationSummary` with counts for initial/final rows, removed variables, detected windows, and discarded rows. The test `test_configurable_preparation_pipeline_respects_stages` validates that both stages respond correctly to their UI toggles.

1.6 Feature selection

The selection stage uses `compute_abs_correlations()`, `drop_highly_correlated_features()`, and `rank_features_random_forest()`. Operationally, it runs in two steps:

1. remove highly correlated variables using a configurable threshold;
2. train a Random Forest to rank the remaining variables and keep the effective top- N for experiments.

The same tab generates the Figure 5 replica (density of $|\rho|$) and the Figure 6 replica (top feature importances), and it persists the selection payload inside the feature DuckDB. The test `test_feature_selection_payload_persists_with_feature_duckdb` covers this persistence path.

1.7 Experimental design and strategies

Available strategies are defined by `EXPERIMENT_STRATEGIES`: `static`, `period_aligned`, `cumulative`, `adaptive_adwin`, `adaptive_arf`, and `adaptive_kswin`. Batch models supported through `MODEL_NAMES` are `XGBoost`, `AdaBoost`, `Random Forest`, and `NNet`; online variants are `ARFmoderate`, `ARFmaj`, `ARFfast`, `KSWINpaper`, and `KSWINseasonal`.

Formal experimental setup Let $Y_0 < Y_1 < \dots < Y_J$ denote the sequence of years observed in the dataset once it is sorted by `interval_start`. The app uses Y_0 as the default base year unless the user overrides it. For each interval t , the pair (x_t, y_t) is formed by the selected feature vector and the one-step-ahead accident target. The experiment always runs on the dataset already cleaned by Stage 1 and Stage 2.

The main orchestrator `run_recalibration_experiments()` builds experiment blocks over the Cartesian product of:

- strategies;
- batch models or online variants;
- balance modes `none`/`smote` whenever applicable;
- sequential repetition seeds.

Therefore, the final output is not a single run but a collection of persisted blocks that are assembled at the end. Formally, if B is the set of experiment blocks and R the set of seeds, the base number of executions is $|B| \times |R|$, plus the canonical global tuning tasks.

Internal batch training and threshold calibration Batch strategies, `adaptive_adwin`, and `adaptive_kswin` all reuse `train_model_with_internal_validation()`. Given a training split T , the actual code executes:

1. a stratified internal split $(T^{\text{fit}}, T^{\text{val}})$ with validation proportion $\rho = 0.2$ by default;
2. hyperparameter tuning on T^{fit} with stratified cross-validated AUC;
3. threshold calibration on the full T using out-of-fold scores;
4. final refit on all rows in T , with optional SMOTE.

Let $\Theta_{m,b}$ be the search space associated with model m and balance mode b . Hyperparameter selection solves

$$\hat{\theta} = \arg \max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^K \text{AUC}_k(\theta),$$

where K is the effective number of stratified folds. If `balance_mode = smote`, the search space itself includes `sampling_strategy` and `k_neighbors`; if `balance_mode = none`, only model hyperparameters are tuned.

Once $\hat{\theta}$ is fixed, the code generates out-of-fold scores s_i^{cv} for every row $i \in T$ and selects the operating threshold through the Youden criterion:

$$\hat{\tau} = \arg \max_{\tau} [\text{TPR}(\tau) - \text{FPR}(\tau)] = \arg \max_{\tau} [\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1].$$

Hard predictions are defined by

$$\hat{y}_i = \mathbf{1}\{s_i \geq \hat{\tau}\}.$$

If out-of-fold calibration becomes ill-posed, the code falls back to the internal holdout T^{val} .

Balancing with SMOTE The `smote` mode is never applied to the test year or to the evaluation stream; it is restricted to training folds and the final refit set. If the minority class has fewer than two observations, or if the minority/majority ratio is already above the requested target, balancing is skipped. Otherwise, SMOTE builds a balanced set $(X^{\text{bal}}, y^{\text{bal}})$ satisfying

$$\frac{N_{\text{minor}}^{\text{new}}}{N_{\text{major}}} \approx \text{sampling_strategy},$$

with an effective neighborhood

$$k_{\text{eff}} = \min(k_{\text{neighbors}}, N_{\text{minor}} - 1).$$

SMOTE artifacts may be persisted and reused at block level through `_load_or_create_smote_artifact()`.

Yearly batch strategies `run_yearly_strategy()` implements three training rules for each target year Y_j , $j \geq 1$:

$$\begin{aligned} \mathcal{T}_j^{\text{static}} &= \{t : \text{year}(t) = Y_0\}, \\ \mathcal{T}_j^{\text{period}} &= \{t : \text{year}(t) = Y_j - 1\}, \\ \mathcal{T}_j^{\text{cum}} &= \{t : \text{year}(t) \leq Y_j - 1\}, \end{aligned}$$

while the evaluation set is always

$$\mathcal{E}_j = \{t : \text{year}(t) = Y_j\}.$$

Each output row contains `training_year`, `prediction_year`, `model`, `balance_mode`, performance metrics, the calibrated threshold, training time, and `n_train/n_test`. In the `static` strategy, because $\mathcal{T}_j^{\text{static}}$ is identical for all future years, the implementation caches and reuses the fitted bundle to avoid redundant tuning and refitting; this behavior is validated by `test_run_yearly_strategy_reuses_static_train`.

The expected result of this block is:

- one row in `yearly_results` per (strategy, model, balance, Y_j , seed);
- one ROC payload segment per prediction year in `roc_payload`;
- Appendix Tables A.6, A.7, and A.8 for `static`, `period_aligned`, and `cumulative`, respectively.

Adaptive recalibration with ADWIN `run_adaptive_strategy()` first trains a batch model on the base year Y_0 , then processes the post-base stream chronologically. For each stream row it computes a score s_t , a hard prediction $\hat{y}_t = \mathbf{1}\{s_t \geq \hat{\tau}\}$, and a binary error signal

$$e_t = \mathbf{1}\{\hat{y}_t \neq y_t\}.$$

The `SimpleADWIN` class maintains an error window W . For each candidate split $W = W_0 \cup W_1$, with sizes n_0 and n_1 , it defines

$$\mu_0 = \frac{1}{n_0} \sum_{e \in W_0} e, \quad \mu_1 = \frac{1}{n_1} \sum_{e \in W_1} e,$$

$$m = \left(\frac{1}{n_0} + \frac{1}{n_1} \right)^{-1}, \quad \epsilon = \sqrt{\frac{1}{2m} \log \left(\max \left(1.0000001, \frac{4 \log n}{\delta} \right) \right)},$$

where $n = |W|$ and $\delta = \text{adwin_delta}$. Drift is declared when

$$|\mu_0 - \mu_1| > \epsilon.$$

When that happens, the code:

- closes the accumulated prediction segment up to t ;
- stores `drift_date`, `W`, `W0`, `W1`, and `remaining_periods`;
- attempts recalibration using the last $|W_1|$ stream rows.

Retraining only happens if the W_1 window contains at least $\max(20, \lfloor \text{min_window}/10 \rfloor)$ rows, or the user-specified `min_retrain_size`, and both target classes are present. Otherwise, the segment is still closed, but the current model continues unchanged. The expected result is an `adaptive_results` table with one row per detected drift plus one final segment with `remaining_periods = 0`; those rows feed the ADWIN portion of Appendix A.9.

Adaptive Random Forest `run_arf_strategy()` implements a genuinely online strategy. On the base year Y_0 , `train_arf_with_internal_validation()` separates a trailing chronological validation block and uses the leading block to warm up the ARF ensemble. It then predicts sequentially on the validation block, learns each label immediately, and fixes $\hat{\tau}$ through the same Youden criterion.

During the subsequent stream, the model is updated one observation at a time:

$$s_t = f_{t-1}(x_t), \quad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

where $\mathcal{U}$ is the online `learn_one` update. Unlike ADWIN and KSWIN, there is no external batch retraining after drift: adaptation is internal to the forest. For that reason, the code never duplicates this strategy by `balance_mode`; it always uses `not_applicable`.

Results are segmented by prediction year rather than by externally detected drift. Each expected row contains:

- `prediction_year`;
- `segment_rows`;
- `n_internal_drifts`;
- `n_internal_warnings`;

- performance metrics and the operating threshold.

As a consequence, Appendix A.9 represents ARF as yearly online segments rather than as $W/W_0/W_1$ windows. The tests `test_run_arf_strategy_variants` and `test_recalibration_experiments_support_adaptive` validate this output shape.

Adaptive recalibration with KSWIN `run_kswin_strategy()` clearly separates covariate-drift detection from prediction. It first trains a batch model on Y_0 . It then selects monitored variables with `_select_kswin_monitor_columns()`, which in the current implementation simply takes the first top-k numeric columns from the ordered `feature_cols` list. This is not an online ranking: it is a deterministic choice on top of the already approved feature set.

For each monitored feature f , KSWIN operates in one of two spaces:

- **KSWINpaper**: the raw value $x_{t,f}$;
- **KSWINseasonal**: a seasonal z-score

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

with fallback to hour-level profiles and then to global base-year profiles.

Each detector uses $\alpha = 10^{-4}$, `window_size` = 300, and `stat_size` = 30, so Appendix A.9 reports

$$W = 300, \quad W_0 = 270, \quad W_1 = 30$$

as fixed detector sizes rather than learned adaptive windows.

Let F_t be the set of features whose detector fires at time t . The block-level drift condition is:

$$|F_t| \geq \text{vote_threshold}.$$

When that happens, the prediction segment is closed, `vote_count`, `detected_features`, and `monitored_features` are stored, and a temporal retraining window is selected:

$$\mathcal{R}_t = \{i : t_i \geq t - D\},$$

where $D = \text{kswin_retrain_days}$. If $|\mathcal{R}_t| < \text{kswin_min_retrain_rows}$, the code uses the last `min_rows` observations; if that window still contains a single target class, it expands to all observations seen up to t . After batch refitting, the KSWIN detectors are rebuilt and rewarmed on the retraining window.

The expected result of KSWIN is one row per voted drift plus a final segment, with extra columns such as `vote_threshold`, `monitor_feature_count`, `retrain_rows`, and `retrain_positive_rows`. The tests `test_run_kswin_strategy_variants`, `test_recalibration_experiments_support_adaptive_kswin`, and `test_kswin_optuna_studies_capture_base_and_retrains_with_balance_mode` validate that behavior.

Seeds, global tuning, persistence, and resume `run_recalibration_experiments()` does not tune every batch block from scratch. Before launching blocks, it constructs canonical tuning tasks on the base-year split Y_0 for each (model, balance) combination that will be reused by yearly strategies, ADWIN, or KSWIN. That global tuning is persisted in `tuning_dir` and later injected as `fixed_params/fixed_tuning_artifact` into downstream blocks. Methodologically, this enforces comparability across strategies that share the same model family, because they all start from the same canonical $\hat{\theta}$ per seed and balance mode.

Each block persists:

- yearly or adaptive result rows;
- segmented ROC payload;
- its own `execution_log`;
- references to tuning and SMOTE artifacts.

The central `manifest.json` maintains progress, completed blocks, available caches, and error status. If a run fails midway, the next execution can resume from the checkpoint and reuse already computed tuning/SMOTE artifacts; this is covered by `test_preview_recalibration_checkpoint_detects_resumable_manifest` and `test_recalibration_experiments_resume_from_failed_checkpoint`.

Expected results Methodologically, a completed experiment must produce seven mutually consistent outputs:

- `yearly_results`: one row per yearly batch split, with `strategy`, `prediction_year`, `model`, `balance_mode`, metrics, threshold, and sample sizes;
- `adaptive_results`: one row per adaptive segment, either drift-delimited for ADWIN/KSWIN or year-delimited for ARF;
- `summary`: mean metrics by (`strategy`, `model`, `balance_mode`), including `n_segments` and `n_repetitions`;
- `appendix_tables`: A.6 for `static`, A.7 for `period_aligned`, A.8 for `cumulative`, and A.9 for adaptive results;
- `appendix_tables_mean`: the same tables averaged across sequential seeds, enriched with `seed_list` and `run_orders`;
- `average_roc`: Figure 7 style mean ROC curves, built by interpolating each individual curve on a uniform 101-point FPR grid and averaging TPR:

$$\overline{\text{TPR}}_g(u) = \frac{1}{J_g} \sum_{j=1}^{J_g} \widetilde{\text{TPR}}_{g,j}(u);$$

- `run_manifest` and `execution_log`: reproducible traceability for configuration, seeds, thresholds, tuning, checkpoints, failures, and resource consumption.

The tests `test_end_to_end_recalibration_and_average_roc`, `test_recalibration_experiments_compare_balance` and `test_recalibration_experiments_persist_optuna_json` verify precisely that this result package exists, is internally consistent, and can be persisted as the final JSON artifact.

1.8 Metrics, tables, and final artifacts

Evaluation uses four base metrics computed by `compute_classification_metrics()`: `auc`, `sensitivity`, `specificity`, and `error_rate`, plus the operational `threshold`. On top of those tables, the module builds:

- consolidated summaries with `summarize_results()`;
- Appendix A.6–A.9 tables with `format_appendix_tables()`;

- seed-averaged appendix tables with `format_appendix_tables_mean()`;
- Figure 7 style average ROC curves with `build_average_roc_curves()`.

Primary persistence is organized in three layers:

- exported feature DuckDB files under `Resultados/`;
- per-run checkpoints under `Resultados/drift_recalibration_runs/`, containing `manifest.json`, persisted blocks, tuning cache, and SMOTE artifacts;
- the final results JSON written by `_persist_drift_recalibration_json()` as `drift_recalibration_optuna_*`.

The `run_manifest` stores configuration, seeds, strategies, grid sizes, active features, and checkpoint paths. The `execution_log` captures tuning steps, training events, errors, adaptive retraining, and resource traces.

1.9 Validation through tests

The file `tests/test_drift_detection_app.py` provides broad functional coverage of the pipeline. The most relevant cases include:

- `test_article_coverage_is_100_percent`;
- `test_drift_article_features_cover_table4_and_target_shift`;
- `test_drift_batched_flow_features_duckdb_consistency`;
- `test_end_to_end_recalibration_and_average_roc`;
- `test_recalibration_experiments_compare_balance_modes_end_to_end`;
- `test_recalibration_experiments_persist_optuna_json`;
- `test_recalibration_experiments_resume_from_failed_checkpoint`;
- `test_recalibration_experiments_support_adaptive_arf`;
- `test_recalibration_experiments_support_adaptive_kswin`.

Taken together, these tests show that the *Drift detection* documentation describes repository behavior that already exists, rather than an aspirational design.